PRADITA University

IF140303-Web Application Development

Session-06:

Avatar Generator 2/2

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

PRADITA University

1. Introduction
   - Avatar Generator
   - Avatar Pipeline
   - Avatar Computation
2. Filtering Cells: `remove_odd_cells/1`
3. Generate Pixel Map: `generate_pixel_map/1`
4. Rendering Image: `render_image/1`
5. Storing Image: `store_image/2`
6. Project Configuration: `mix.exs`
7. Project Configuration: `project/0`
8. Application Configuration: `application/0`
9. Dependencies Definition: `deps/0`
10. Managing Dependencies and Running the Project
11. Adjusting `egd` for Erlang/OTP 27

Introduction

PRADITA University

This section explains the development of an **avatar generator** built using the Elixir programming language. The program showcases key Elixir features:

- `defstruct` for defining structured data types,
- `defmodule` for modular organization,
- `Application` behavior for managing runtime lifecycle.

The generator takes a text input (e.g., a username) and computes its **MD5 hash**, which is a **128-bit** (**16-byte**) fixed-length value. **Each byte** is **converted** into an **integer** between **0 and 255**, forming a sequence of numbers that represents the hash in numeric form. These integers are then used to:

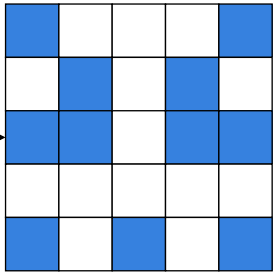
- determine which grid cells will be filled,
- assign colors or symmetry patterns to each region.

Thus, every unique input string produces a distinct yet deterministic avatar.

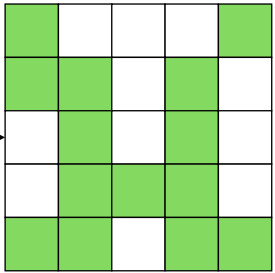
Avatar Generator

PRADITA University

"wolverine" →



"superman" →



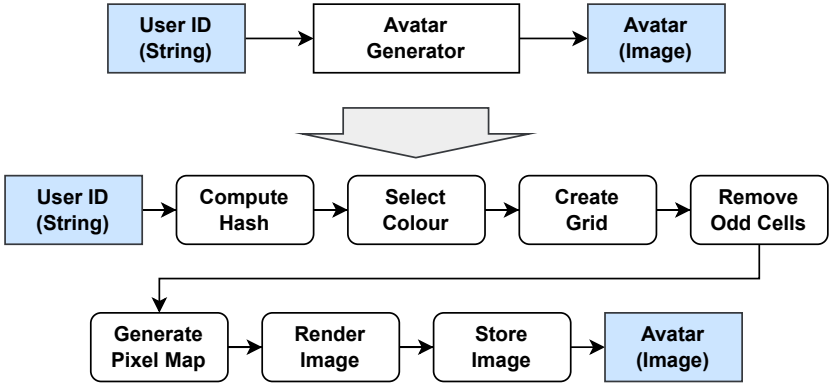
wolverine's md5 hash = [54, 129, 223, 141, 4, 71, 14, 204, 101, 5, 59, 121, 14, 25, 160, 101]

superman's md5 hash = [132, 217, 97, 86, 138, 101, 7, 58, 59, 207, 14, 178, 22, 178, 165, 118]

The 128-bit MD5 hash is split into 16 integers (0–255), each encoding color or grid placement. These values define the avatar's pattern and symmetry, ensuring every input yields a unique, consistent design.

Avatar Pipeline

PRADITA University



Input text is hashed (MD5, 128-bit/16 bytes) and drives color and layout. Functions run sequentially to transform data into a rendered PNG image.

Pipeline Code

PRADITA University

```
compute_hash
|> select_color
|> create_grid
|> remove_odd_cells
|> generate_pixel_map
|> render_image
|> store_image
```

compute_hash: MD5 16 integers (0–255). **select_color**: derives RGB from first bytes. **create_grid**: builds 5×5 layout with symmetry. **remove_odd_cells**: keep even-valued cells. **generate_pixel_map**: map cells to coordinates. **render_image**/**store_image**: draw and save PNG.

Avatar Computation

PRADITA University

- The avatar is generated as a **5×5 grid** image.
- **Columns 4 and 5 mirror columns 2 and 1**, creating symmetry and visual balance.
- **The first three columns** use integer values of the MD5 hash of the `username`.
- **Even** values fill colored cells; **odd** values remain white.
- **The first three hash bytes** define the RGB color used for the avatar.

0 **54**	1 **129**	2 **223**	3 **129**	4 **54**
5 **141**	6 **4**	7 **71**	8 **4**	9 **141**
10 **14**	11 **204**	12 **101**	13 **204**	14 **14**
15 **5**	16 **59**	17 **121**	18 **59**	19 **5**
20 **14**	21 **25**	22 **160**	23 **25**	24 **14**

The first 3 values are the RGB colour

[54, 129, 223, 141, 4, 71, 14, 204, 101, 5, 59, 121, 14, 25, 160, 101]

Figure: Hash integers, mirroring, and RGB mapping.

Filtering Cells: `remove_odd_cells/1`

RADITA niversity

The `remove_odd_cells/1` function filters the avatar grid to retain only even-valued cells. It helps simplify the final pattern and maintain symmetry.

- Uses `Enum.filter/2` to traverse each grid cell.
- Keeps only cells where `rem(code, 2) == 0`.
- Removes odd-numbered cells that do not meet the condition.
- Produces a cleaner, even-coded grid for rendering.

```
1 def remove_odd_cells(%Avatar.Image{grid: grid} = image) do
2   grid = Enum.filter grid, fn({code, _index}) ->
3     rem(code, 2) == 0
4   end
5 end
```

Generate Pixel Map: `generate_pixel_map/1`

The `generate_pixel_map/1` function converts the filtered grid into a pixel map that defines the exact coordinates of each cell on the image canvas. It calculates the `x` and `y` positions based on each cell index, determines the `top_left` and `bottom_right` points for 50×50 pixel blocks, and stores the resulting coordinate pairs in the `pixel_map` field.

```elixir
def generate_pixel_map(%Avatar.Image{grid: grid} = image) do
  pixel_map = Enum.map grid, fn({_code, index}) ->
    x = rem(index, 5) * 50
    y = div(index, 5) * 50

    top_left = {x, y}
    bottom_right = {x + 50, y + 50}

    {top_left, bottom_right}
  end

  %Avatar.Image{image | pixel_map: pixel_map}
end
```

Rendering Image: `render_image/1`

PRADITA University

The `render_image/1` function generates the final avatar image from the pixel map and color data. It uses the Erlang `:egd` module to create a 250×250 canvas, apply the selected RGB color, and draw filled rectangles for each cell in the pixel map before rendering the complete image.

```elixir
def render_image(%Avatar.Image{color: color, pixel_map: pixel_map}) do
  image = :egd.create(250, 250)
  fill = :egd.color(color)

  Enum.each pixel_map, fn({start, stop}) ->
    :egd.filledRectangle(image, start, stop, fill)
  end

  :egd.render(image)
end
```

Storing Image: `store_image/2`

PRADITA University

The `store_image/2` function saves the rendered image as a `.png` file. It takes two arguments — the image binary and the original `input` string — which is used to name the output file. The built-in `File` module writes the binary image data directly to disk.

```elixir
def store_image(image, input) do
  File.write("#{input}_avatar.png", image)
end
```

Project Configuration: `mix.exs`

PRADITA University

The `mix.exs` file defines essential project settings in the `AvatarGenerator.MixProject` module. It contains metadata, dependencies, and application configuration generated automatically by `mix new`. The module uses `Mix.Project` to manage build, compile, and dependency tasks for the Elixir project.

```elixir
defmodule AvatarGenerator.MixProject do
  use Mix.Project
end
```

Project Configuration: `project/0`

PRADITA University

The `project/0` function defines essential project metadata used by Mix for building and running the app. It returns a keyword list that specifies the application name, version, Elixir requirement, build mode, and dependencies.

```elixir
def project do
  [
    app: :avatar_generator,
    version: "0.1.0",
    elixir: "~> 1.17",
    build_embedded: Mix.env == :prod,
    start_permanent: Mix.env() == :prod,
    deps: deps()
  ]
end
```

This configuration ensures that the project is properly defined and can be compiled, run, and packaged according to Mix standards.

Application Configuration: `application/0`

The `application/0` function defines runtime settings for the project. It specifies which additional applications to start and which main module serves as the entry point.

```elixir
def application do
  [
    extra_applications: [:logger],
    mod: {AvatarGenerator, []}
  ]
end
```

This configuration ensures that the logger is available at runtime and that the `AvatarGenerator` module is automatically started when the application runs.

Dependencies Definition: `deps/0`

PRADITA University

The `deps/0` function lists external dependencies required by the application. Dependencies can come from Hex.pm packages or GitHub repositories.

```
defp deps do
  [
    {:egd, github: "erlang/egd", manager: :rebar3}
    # {:dep_from_hexpm, "~> 0.3.0"},
    # {:dep_from_git, git: "https://github.com/elixir-lang/my_dep.git", tag:
        "0.1.0"}
  ]
end
```

Here, the main dependency is `:egd`, a graphics library fetched from GitHub and managed using `rebar3`. This setup allows the project to handle image rendering functionality.

Cleaning and Fetching Dependencies

PRADITA University

Before compiling or running the project, dependencies must be managed properly. To remove all existing dependencies and clear cache files, use:

```
$ mix deps.clean --all
```

Then, download the required libraries as defined in `deps/0` using:

```
$ mix deps.get
```

These commands ensure that all dependencies are refreshed and correctly installed according to the configuration in `mix.exs`.

Compiling and Running the Project

PRADITA University

After dependencies are set, compile the project and run it using Mix commands.
To compile the source code:

```
$ mix compile
```

To execute the main application:

```
$ mix run
```

Or to run a specific function:

```
$ mix run -e "AvatarGenerator.generate('example')"
```

If dependency issues occur, update them with:

```
$ mix deps.update --all
```

These steps ensure a smooth workflow for compiling, updating, and running Elixir projects.

Issue with `zlib:crc32/2` Removal

PRADITA University

In Erlang/OTP 27, the function `zlib:crc32/2` was removed, causing compatibility issues with the `egd` library used in this project. This results in an `undefined function` error during compilation or runtime. To fix it, the `egd` source code must be updated to use the new function `erlang:crc32/2` instead.

Modifying the `egd_png.erl` File

PRADITA University

After running `mix deps.get`, open `deps/egd/src/egd_png.erl`. Locate the function `create_chunk/2` and replace the removed call `zlib:crc32/2` with `erlang:crc32/2` as shown below.

Original:

```
create_chunk(Bin, Z) when is_binary(Bin) ->
  Sz = size(Bin)-4,
  Crc = zlib:crc32(Z, Bin),
  <<Sz:32, Bin/binary, Crc:32>>.
```

Updated:

```
create_chunk(Bin, _Z) when is_binary(Bin) ->
  Sz = size(Bin)-4,
  % Crc = zlib:crc32(Z,Bin),
  Crc = erlang:crc32(Bin),
  <<Sz:32,Bin/binary,Crc:32>>.
```

Recompiling the egd Library

PRADITA University

Once the modification is complete, recompile the egd library so the new code takes effect. Use the following command:

```
$ mix deps.compile egd
```

This command rebuilds the egd dependency with the updated code, ensuring it functions properly with Erlang/OTP 27.

Running the Project After Fix

PRADITA University

After recompiling, recompile all dependencies and run the project again to confirm that the issue has been resolved.

```
$ mix deps.compile
$ mix run
```

If all steps are followed correctly, the project will run without errors. The fix replaces the deprecated `zlib:crc32/2` with `erlang:crc32/2`, ensuring full compatibility with Erlang/OTP 27. The generated avatar, such as for `"wolverine"`, will be saved as `wolverine_avatar.png`.